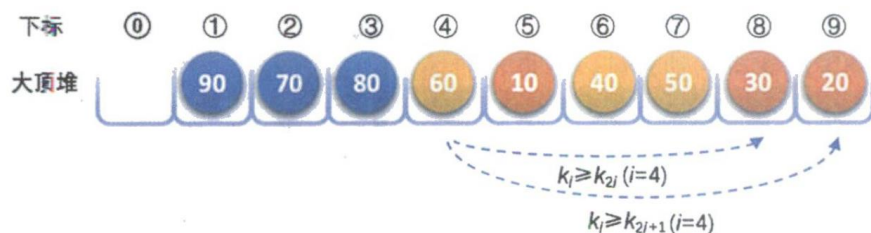
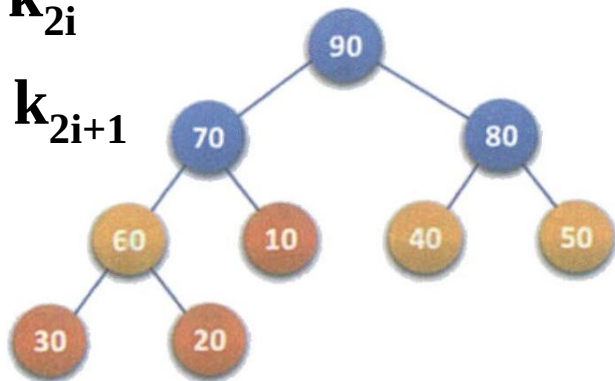


回顾

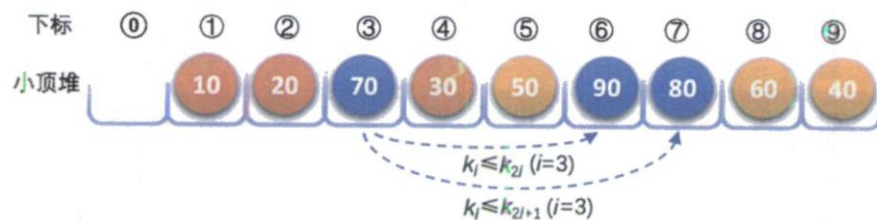
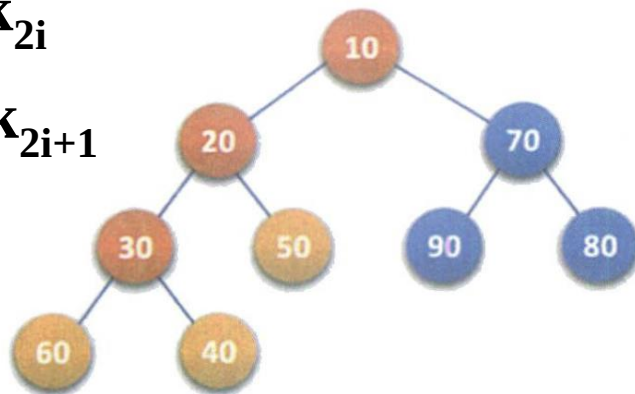
□ 堆排序

堆是具有以下性质的完全二叉树：每个结点的值都大于或等于其左右孩子结点的值，称为大顶堆；或者每个结点的值都小于或等于其左右孩子结点的值，称为小顶堆。

$$\begin{cases} k_i \geq k_{2i} \\ k_i \geq k_{2i+1} \end{cases}$$



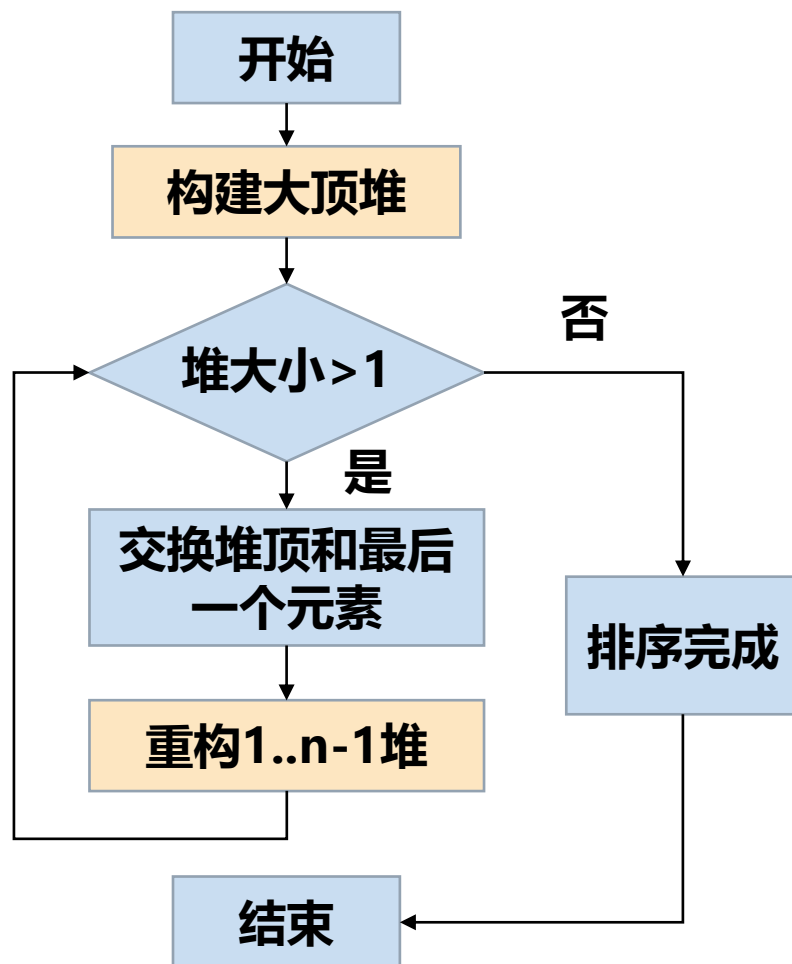
$$\begin{cases} k_i \leq k_{2i} \\ k_i \leq k_{2i+1} \end{cases}$$



回顾

□ 堆排序

- ① 首先将待排序的数组构造成一个大根堆，此时，整个数组的最大值就是堆结构的顶端
- ② 将顶端的数与末尾的数交换，此时，末尾的数为最大值，剩余待排序数组个数为 $n-1$
- ③ 将剩余的 $n-1$ 个数再构造成大根堆，再将顶端数与 $n-1$ 位置的数交换，如此反复执行，便能得到有序数组



第10章 内部排序

10.1 概述

10.2 插入排序

10.3 交换排序

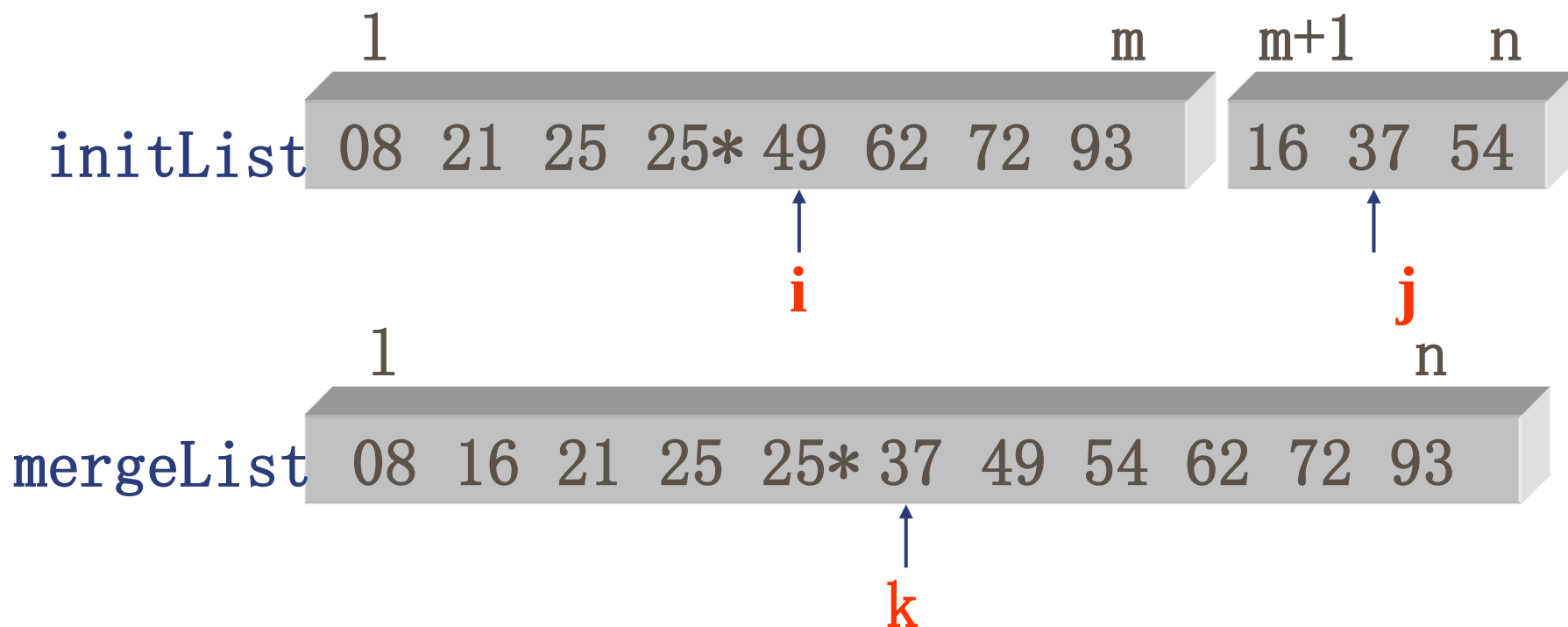
10.4 选择排序

10.5 归并排序

10.6 基数排序

10.5 归并排序

对象序列 **initList** 中有两个有序表 $V[1] \cdots V[m]$ 和 $V[m+1] \cdots V[n]$ 。它们可归并成一个有序表，存于另一对象序列 **mergedList** 的 $V[1] \cdots V[n]$ 中。



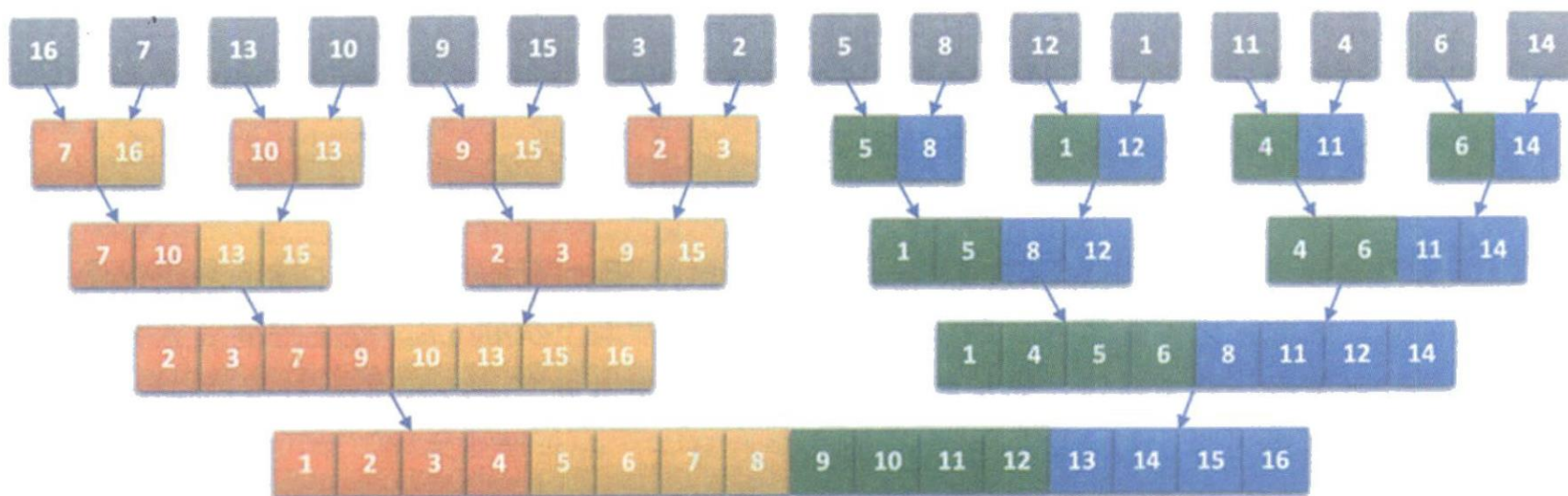
10.5 归并排序

归并排序的基本思想是：将两个（或以上）的有序表组成新的有序表。

更实际的意义：可以把一个长度为 n 的无序序列看成是 n 个长度为1的有序子序列，首先做两两归并，得到 $\lceil n/2 \rceil$ 个长度为2的有序子序列；再做两两归并， $\dots$ ，如此重复，直到最后得到一个长度为 n 的有序序列。

10.5 归并排序

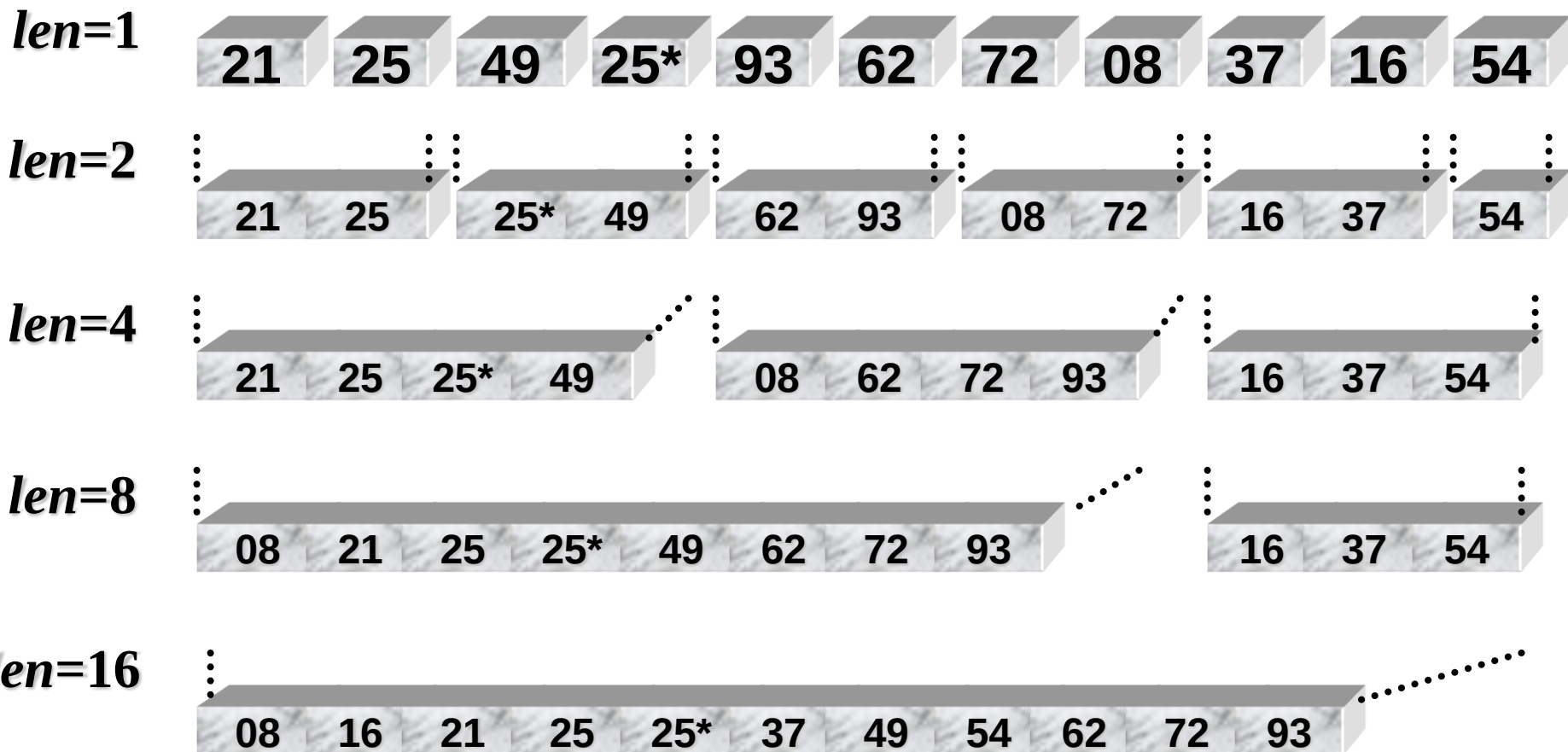
{16, 7, 13, 10, 9, 15, 3, 2, 5, 8, 12, 1, 11, 4, 6, 14}



10.5 归并排序

例：关键字序列 $T = (21, 25, 49, 25^*, 93, 62, 72, 08, 37, 16, 54)$ ，请给出归并排序的具体实现过程。

10.5 归并排序



整个归并排序仅需 $\lceil \log_2 n \rceil$ 趟

10.5 归并排序

```
void Merge(int S[], int &T[],int i,int m,int n) {  
    /*i为第一个子表起始下标， m为第一个子表最后一个结点下标*/  
    int j,k; //k为合并表T的下标  
    for(j=m+1,k=i;i<=m&&j<=n;k++) {  
        if(S[i]<S[j]) T[k]=S[i++];  
        else T[k]=S[j++];  
    }  
    if(i<=m) { //将剩余的S[i..m]合并到T中  
        for(;i<=m;i++) T[k++]=S[i];  
    }  
    if(j<=n) { //将剩余的S[j..n]合并到T中  
        for(;j<=n;j++) T[k++]=S[j];  
    }  
}
```

10.5 归并排序

```
void MSort(int SR[], int &TR1[], int s, int t){
    //将SR[s...t]归并到TR1[s...t]
    int TR2[MAXSIZE+1];
    if (s==t)
        TR1[s]=SR[s];
    else {
        int m=(s+t)/2; // 将SR[s..t]平分为 SR[s..m]和SR[m+1..t]
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t); // 将TR2[s..m]和TR2[m+1..t]归并到TR1[s..t]
    }
}

void MergeSort(SqList &L){
    MSort(L.r,L.r,1,L.length);
}
```

```
typedef struct seqlist
{
    int r[MAXSIZE];
    int length;
}SqList;
```

10.5 归并排序

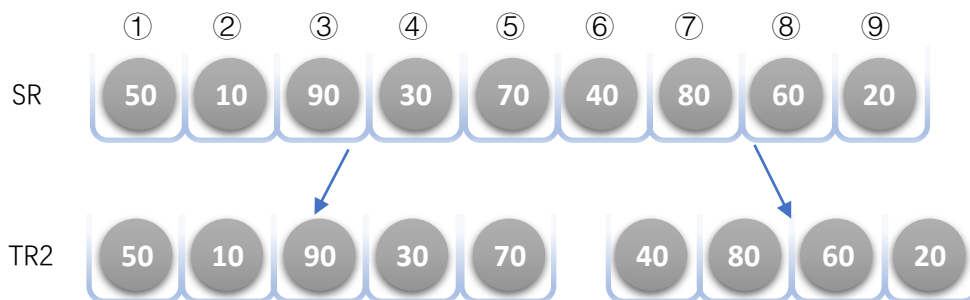
关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int &TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化

s=1, t=9, m=5



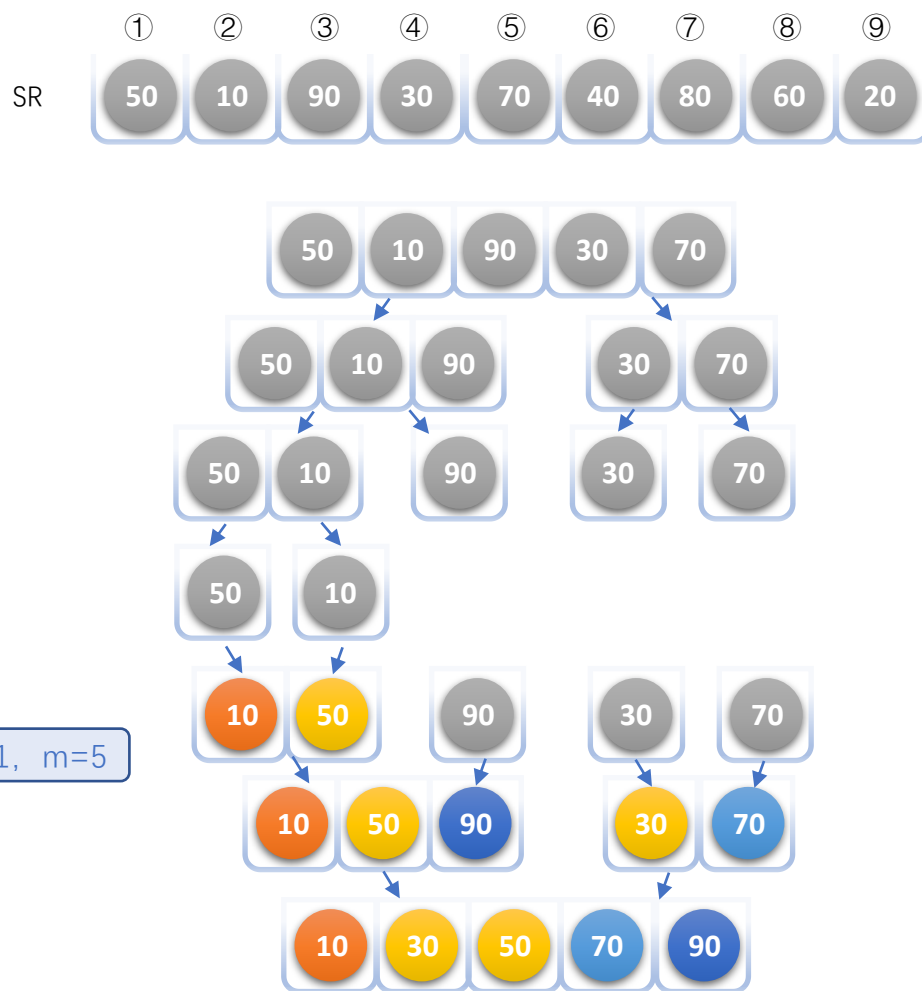
10.5 归并排序

关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int &TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化



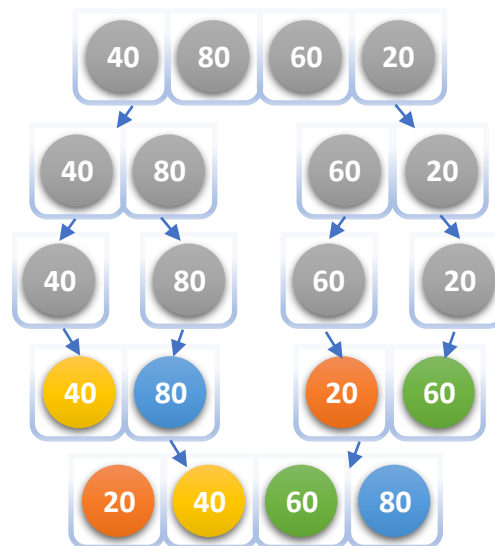
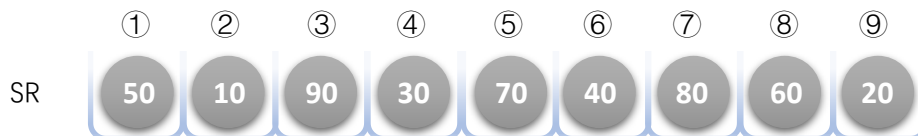
10.5 归并排序

关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int &TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化



m+1=6, t=9

10.5 归并排序

关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int &TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化



s=1, m=5, t=9

10.5 归并排序

关键代码

```
void Merge(int SR[],int &TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=1, m=5, n=9



10.5 归并排序

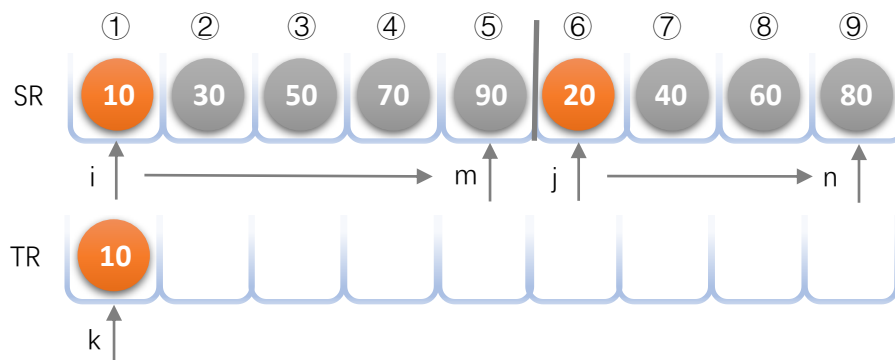
关键代码

```
void Merge(int SR[],int &TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=1, m=5, n=9, j=6, k=1

SR[i]=10
SR[j]=20
TR[k]=10



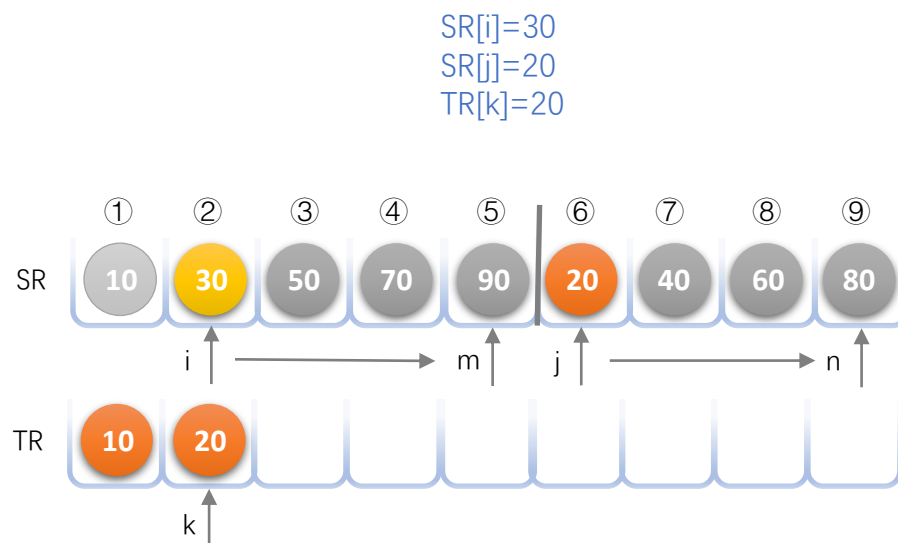
10.5 归并排序

关键代码

```
void Merge(int SR[],int &TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=2, m=5, n=9, j=6, k=2



10.5 归并排序

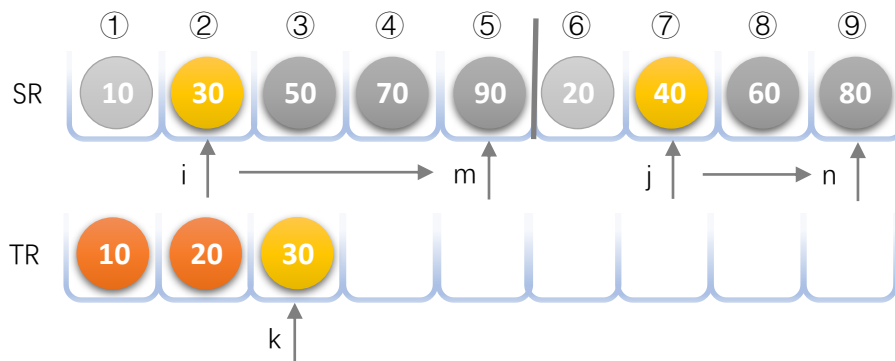
关键代码

```
void Merge(int SR[],int &TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=2, m=5, n=9, j=7, k=3

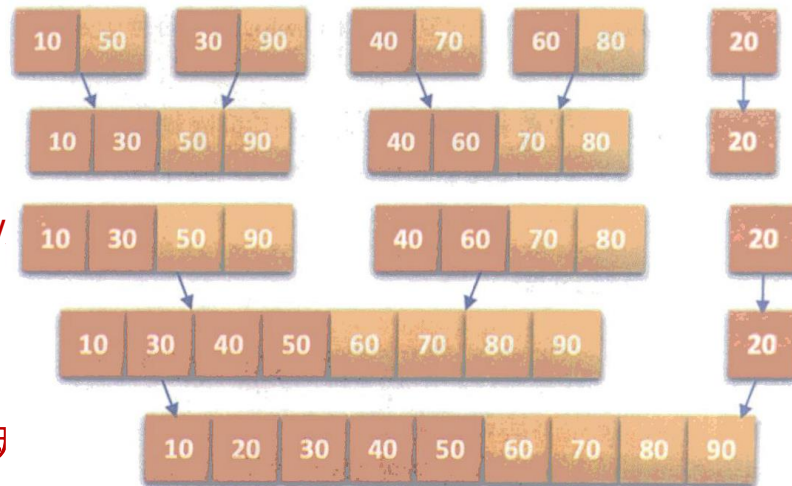
SR[i]=30
SR[j]=40
TR[k]=30



10.5 归并排序

归并排序非递归算法

```
void MergeSort2(SeqList &L) {  
    int k=1;  
    int *T=(int *)malloc(sizeof(int)L.length);  
    if(!T) {  
        printf("No enough memory!\n");  
        exit(-1);  
    }  
    while(k<L.length) {  
        MergePass(L.r,T,k,L.length); //  
        k=k*2; //子序列加倍  
        MergePass(T,L.r,k,L.length);  
        k=k*2; //数将T中已经两  
    }  
}
```



10.5 归并排序

```
void MergePass(int S[],int T[],int s,int n) {
```

```
    //将S[]中长度为s的子序列两两归并到T[]
```

```
    int i=1, j;
```

```
    while(i<=n-2*s+1){ //n-(n-2s+1)+1=2s 说明下标i后面有两个子表, 长度为s
```

```
        Merge(S,T,i,i+s-1,i+2*s-1); //两两归并
```

```
        i=i+2*s;
```

```
    }
```

```
    if(i<n-s+1) //归并最后两个序列
```

// $n-(n-s+1)+1=s$, 由于i小于n-s+1, 所以包括i结点本身至少s+1个结点, 有两个子表, 第二个子表长度小于s

```
        Merge(S,T,i,i+s-1,n);
```

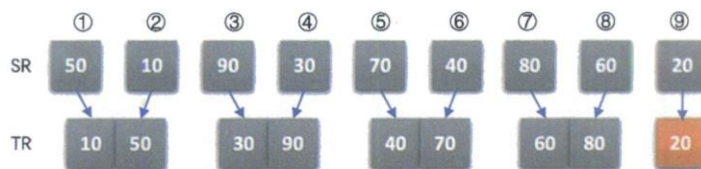
```
    else { //若最后只剩下单个子序列
```

```
        for(j=i;j<=n;j++)
```

```
            T[j]=S[j];
```

```
    }
```

```
}
```

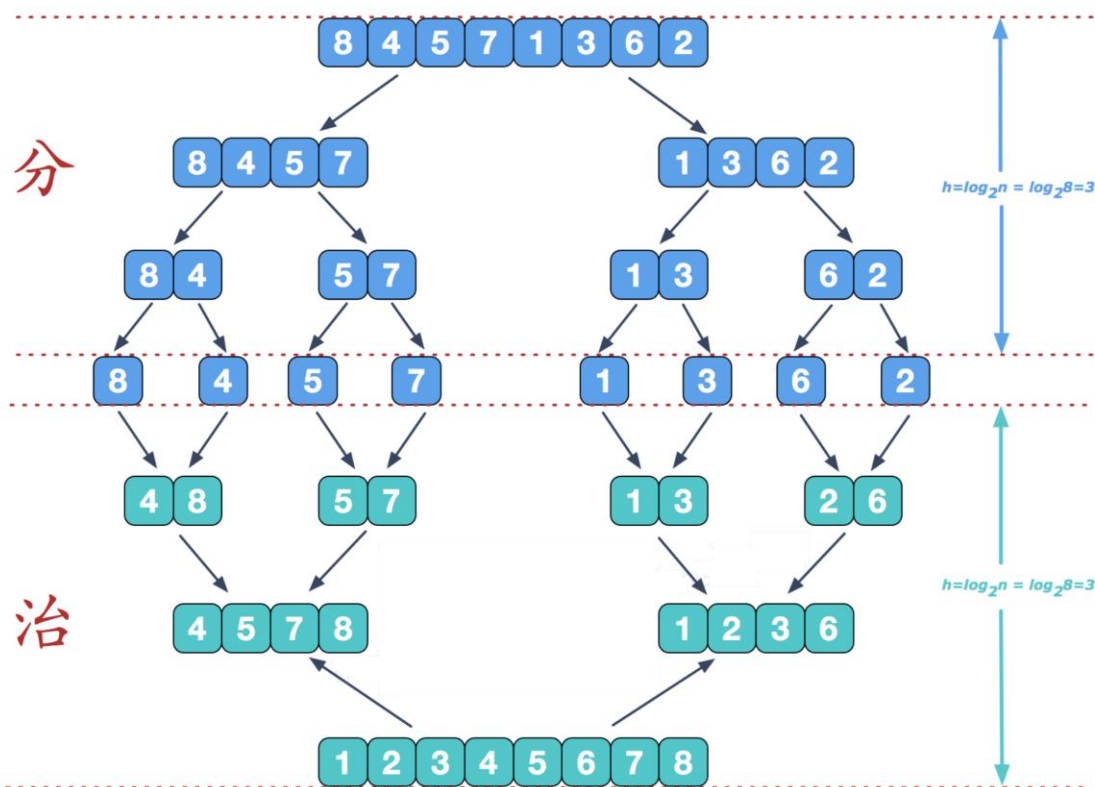


10.5 归并排序

□ 归并排序

归并排序是将两个（或两个以上）有序表合并成一个新的有序表，即把待排序序列分为若干个子序列，每个子序列是有序的。然后再把有序子序列合并为整体有序序列

- ① 把长度为n的输入序列分成两个长度为n/2的子序列
- ② 对这两个子序列分别采用归并排序
- ③ 将两个排序好的子序列合并成一个最终的排序序列



10.5 归并排序

算法分析:

- 时间效率: $O(n\log_2 n)$

因为在递归的归并排序算法中，函数Merge()做一趟两路归并排序，需要调用merge()函数 $\lceil n/(2h) \rceil \approx O(n/h)$ 次，而每次merge()要执行比较 $O(h)$ 次，另外整个归并过程有 $\lceil \log_2 n \rceil$ “层”，所以算法总的时间复杂度为 $O(n\log_2 n)$ 。

10.5 归并排序

算法分析:

- 时间效率: $O(n\log_2 n)$

空间效率: $O(n)$

因为需要一个与原始序列同样大小的辅助序列 (TR) 。这正是此算法的缺点。

- 稳定性: 稳定

第10章 内部排序

10.1 概述

10.2 插入排序

10.3 交换排序

10.4 选择排序

10.5 归并排序

10.6 基数排序

10.6 基数排序

基数排序的基本思想是：

借助多关键字排序的思想对单逻辑关键字进行排序。

即：用关键字**不同的位值**进行排序。

要讨论的问题：

1. 什么是“**多关键字**”排序？实现方法？
2. 单逻辑关键字怎样“**按位值**”排序？

10.6 基数排序

1. 什么是“多关键字”排序？实现方法？

例1：对一副扑克牌该如何排序？

若规定花色和面值的顺序关系为：

花色： $\spadesuit < \clubsuit < \heartsuit < \diamondsuit$

面值： $2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A$

则可以**先按花色**排序，花色相同者**再按面值**排序；

也可以先按面值排序，面值相同者再按花色排序。

10.6 基数排序

1. 什么是“多关键字”排序？实现方法？

例2：职工分房该如何排序？

学校规定：先以总分排序（职称分+工龄分）；

总分相同者，再按配偶总分排序，其次按配偶职称、工龄、人口……等等排序。

以上两例都是典型的多关键字排序！

10.6 基数排序

一般情况下，假定有一个 n 个对象的序列 $\{V_0, V_1, \dots, V_{n-1}\}$ ，且每个对象 V_i 中含有 d 个关键字

如果对于序列中任意两个对象 V_i 和 V_j ($0 \leq i < j \leq n-1$) 都满足：

$$(K_i^1, K_i^2, \dots, K_i^d)$$

$$(K_i^1, K_i^2, \dots, K_i^d) < (K_j^1, K_j^2, \dots, K_j^d)$$

则称序列对关键字 $(K^1, K^2, \dots, K^d)$ 有序。其中， K^1 称为最高位关键字， K^d 称为最低位关键字。

如果关键字是由多个数据项组成的数据项组，则依据它进行排序时就需要利用多关键字排序。

10.6 基数排序

多关键字排序的实现方法通常有两种：

- 最高位优先法**MSD (Most Significant Digit first)**
- 最低位优先法**LSD (Least Significant Digit first)**

排序后形成的有序序列叫做词典有序序列。

10.6 基数排序

- 最高位优先法通常是一个递归的过程：
 - 先根据最高位关键字 K^1 排序,得到若干对象组,对象组中每个对象都有相同关键字 K^1
 - 再分别对每组中对象根据关键字 K^2 进行排序,按 K^2 值的不同,再分成若干个更小的子组,每个子组中的对象具有相同的 K^1 和 K^2 值。
 - 依此重复,直到对关键字 K^d 完成排序为止。
 - 最后,把所有子组中的对象依次连接起来,就得到一个有序的对象序列。

10.6 基数排序

- 最低位优先法

- 首先依据最低位关键字 K^d 对所有对象进行一趟排序
- 再依据次低位关键字 K^{d-1} 对上一趟排序的结果再排序
- 依次重复，直到依据关键字 K^1 最后一趟排序完成，就可以得到一个有序的序列。

使用这种排序方法对每一个关键字进行排序时，不需要再分组，而是整个对象组都参加排序。

10.6 基数排序

例：对一副扑克牌该如何排序？

答：若规定花色为第一关键字（高位），面值为第二关键字（低位），则使用MSD和LSD方法都可以达到排序目的。

10.6 基数排序

MSD方法的思路：先设立4个花色“箱”，将全部牌按花色分别归入4个箱内（每个箱中有13张牌）；然后对每个箱中的牌按面值进行插入排序（或其它稳定算法）。

LSD方法的思路：先按面值分成13堆（每堆4张牌），然后对每堆中的牌按花色进行排序（用插入排序等稳定的算法）。

想一想：用哪种方法更快些？

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

思路： 设 n 个记录的序列为： $\{V_0, V_1, \dots, V_{n-1}\}$ ，可以把每个记录 V_i 的**单关键码** K_i 看成是一个 d 元组 $(K_i^1, K_i^2, \dots, K_i^d)$ ，则其中的每一个分量 K_i^j ($1 \leq j \leq d$) 也可看成是一个关键字。

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

注1: K_i^1 = 最高位, K_i^d = 最低位; K_i 共有 d 位, 可看成 d 元组;

注2: 每个分量 K_i^j ($1 \leq j \leq d$) 有 $radix$ 种取值, 则称 $radix$ 为基数。

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

例1： 关键码984可以看成是 3 元组； 基数 $radix$ 为 10。

(0, 1, ..., 9)

(9, 8, 4)

例2： 关键码dian可以看成是 4 元组； 基数 $radix$ 为 26。

(d, i, a, n)

(a, b, ..., z)

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

614 738 921 485 637 101 215 530 790 306

第一趟分配（按个位排）



第一趟收集

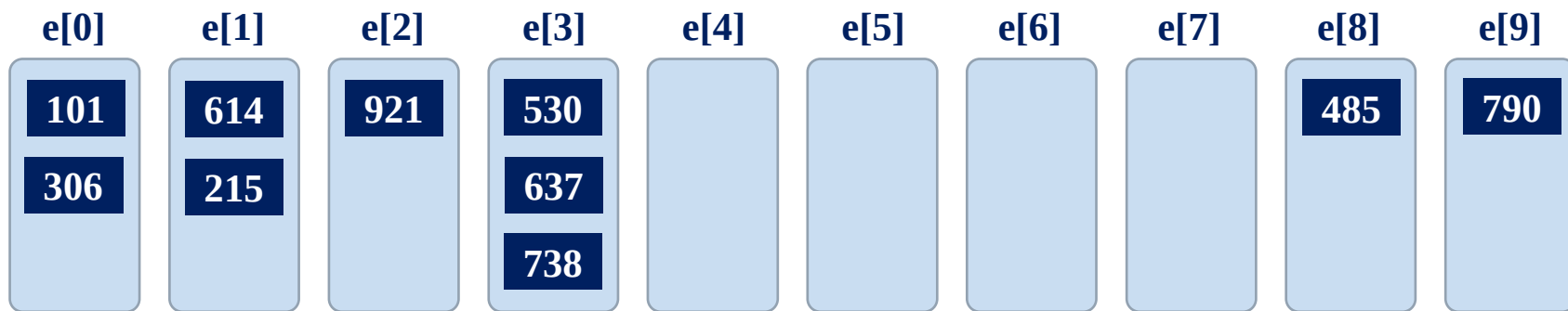
530 790 921 101 614 485 215 306 637 738

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

第一趟：**530** **790** **921** **101** **614** **485** **215** **306** **637** **738**

第二趟分配（按十位排）



第二趟收集

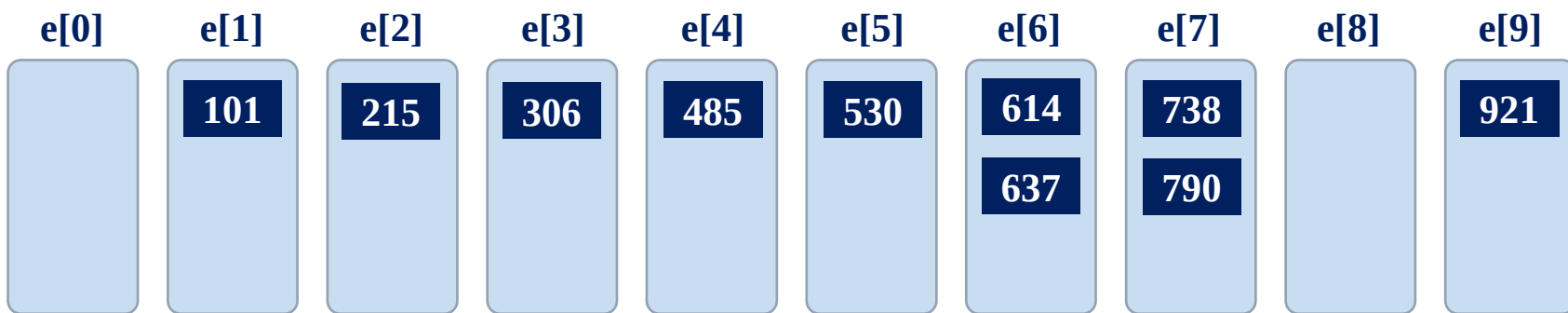
101 **306** **614** **215** **921** **530** **637** **738** **485** **790**

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

第二趟：**101** **306** **614** **215** **921** **530** **637** **738** **485** **790**

第三趟分配（按百位排）



第三趟收集

101 **215** **306** **485** **530** **614** **637** **738** **790** **921**

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

讨论：是借用MSD方式来排序呢，还是借用LSD方式？

例：初始关键字序列 $T = (32, 13, 27, 32^*, 19, 33)$ ，请分别用MSD和LSD进行排序，并讨论其优缺点。

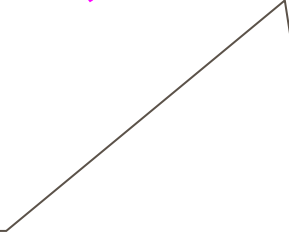
10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

法1 (MSD) : 原始序列: 32, 13, 27, 32*, 19, 33

先按高位 K_i^1 排序: (13, 19), 27, (32, 32*, 33)

再按低位 K_i^2 排序: 13, 19, 27, 32, 32*, 33



因为

有分组，故此算法需递归实现。

10.6 基数排序

2. 单逻辑关键字怎样“按位值”排序？

法2 (LSD) : 原始序列: 32, 13, 27, 32*, 19, 33

先按低位 K_i^2 排序: 32, 32*, 13, 33, 27, 19

再按高位 K_i^1 排序: 13, 19, 27, 32, 32*, 33

无需分组，易编程实现！

10.6 基数排序

计算机怎样实现LSD算法？

例：T= (02, 77, 70, 54, 64, 21, 55, 11) ，用LSD排序。

分析：

①各关键字可视为2元组；②每位的取值范围是：0-9；即基数 $radix = 10$ 。因此，特设置10个队列，并编号为0-9。

10.6 基数排序

计算机怎样实现LSD算法？

原始序列

1	02
2	77
3	70
4	54
5	64
6	21
7	55
8	11

先对低位扫描

分配
过程

又称散列过程！

10个队列

0	70
1	21, 11
2	02
3	
4	54, 64
5	55
6	
7	77
8	
9	

出队

收集
过程

出队后序列

1	70
2	21
3	11
4	02
5	54
6	64
7	55
8	77

下一步

10.6 基数排序

计算机怎样实现LSD算法？

出队后序列

1	70
2	21
3	11
4	02
5	54
6	64
7	55
8	77

再对高位扫描



再次
分配

再次入队

0	02
1	11
2	21
3	
4	
5	54, 55
6	64
7	70, 77
8	
9	

再次出队后序列

再次出队



再次
收集

02
11
21
54
55
64
70
77

10.6 基数排序

计算机怎样实现LSD算法？

小结：排序时经过了反复的“分配”和“收集”过程。当对关键字所有的位进行扫描排序后，整个序列便从无序变为有序了。

10.6 基数排序

效率分析

时间效率: $O(d \cdot (n + rd))$

(d :关键字个数, rd :关键字取值范围为 rd 个值)

空间效率: $O(d + rd)$

稳定性: 稳定

10.6 基数排序

例如：10000个人按照生日排序

年 月 日 3个关键字

假设取值范围分别是：1934~2022年 1~12月 1~31日

$$O(n^2) \approx 10^8$$

$$O(n \log n) \approx 10^5$$

$$\begin{aligned} O(k*(n+m)) &\approx (10000+89) + (10000+12) + (10000+31) \\ &\approx 10^4 \end{aligned}$$

10.6 基数排序

讨论：所用队列是顺序结构，浪费空间，能否改用链式结构？

能！

用链队列来实现基数排序——

链式基数排序

10.6 基数排序

链式基数排序实现思路：

针对 d 元组中的每一位分量，把原始链表中的所有记录，按 K_j 的取值，让 $j = d, d-1, \dots, 2, 1$,

- ① 先“分配”到 $radix$ 个链队列中去；
- ② 然后再按各链队列的顺序，依次把记录从链队列中“收集”起来；
- ③ 分别用这种“分配”、“收集”的运算逐趟进行排序；
- ④ 在最后一趟“分配”、“收集”完成后，所有记录就按其关键码的值从小到大排好序了。

10.6 基数排序

链式基数排序

例: 请实现以下关键字序列的链式基数排序:

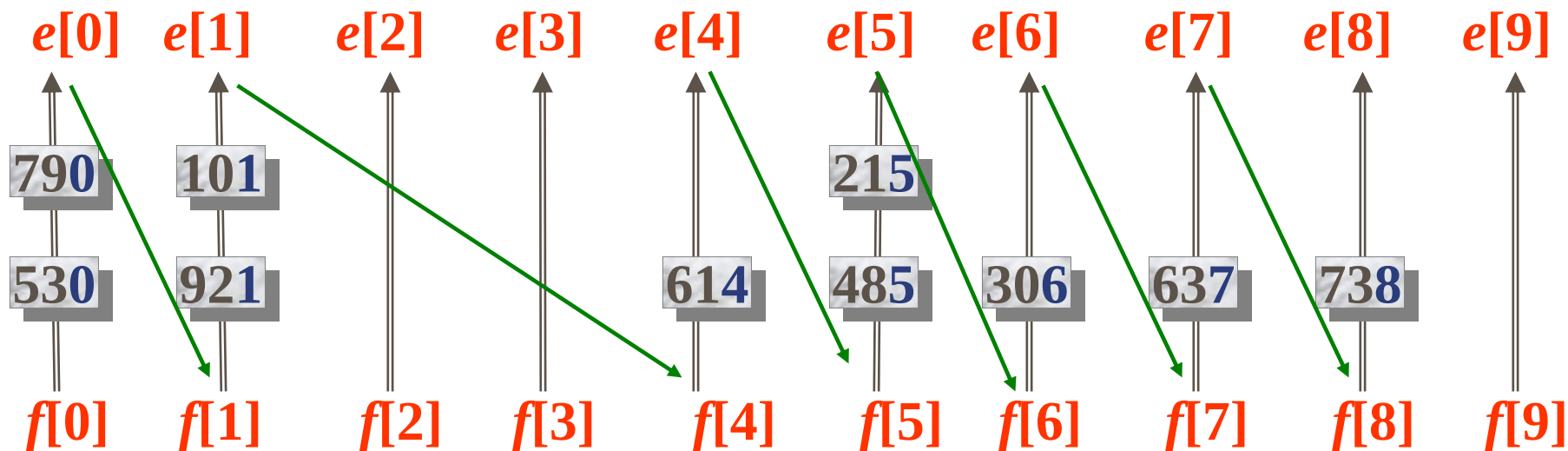
T= (614, 738, 921, 485, 637, 101, 215,
530, 790, 306)

10.6 基数排序

原始序列静态链表:



第一趟分配 (从最低位 $i=3$ 开始排序, $f[]$ 是队首指针, $e[]$ 为队尾指针)



第一趟收集 (让队尾指针 $e[i]$ 链接到下一非空队首指针 $f[i+1]$ 即可)

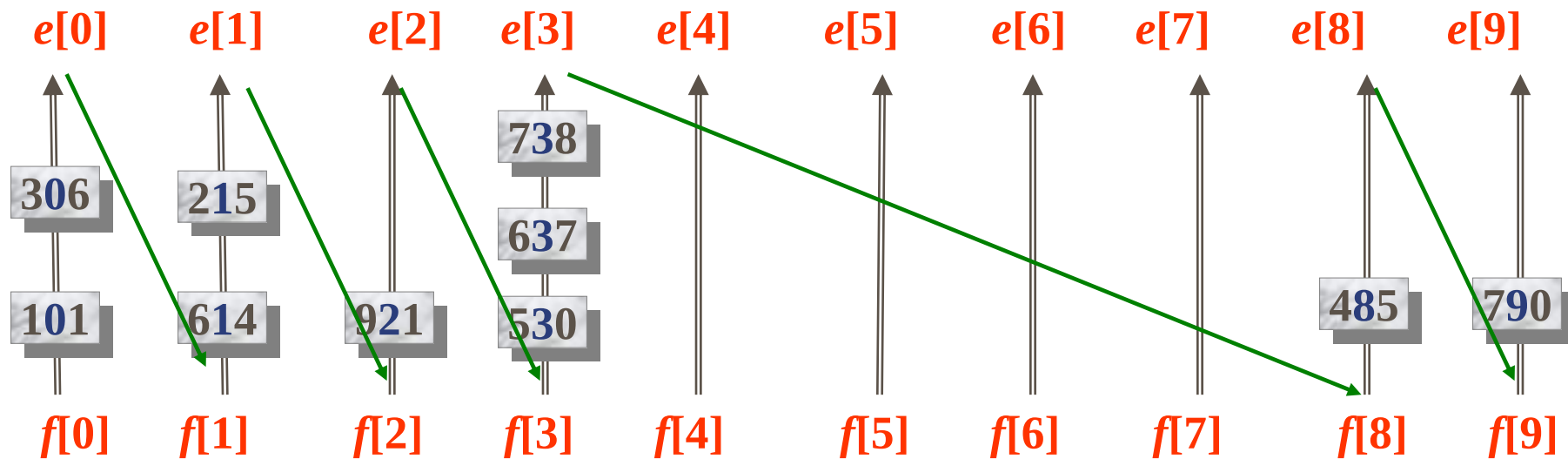


10.6 基数排序

第一趟收集的结果:



第二趟分配 (按次低位 $i=2$)



第二趟收集 (让队尾指针 $e[i]$ 链接到下一非空队首指针 $f[i+1]$)

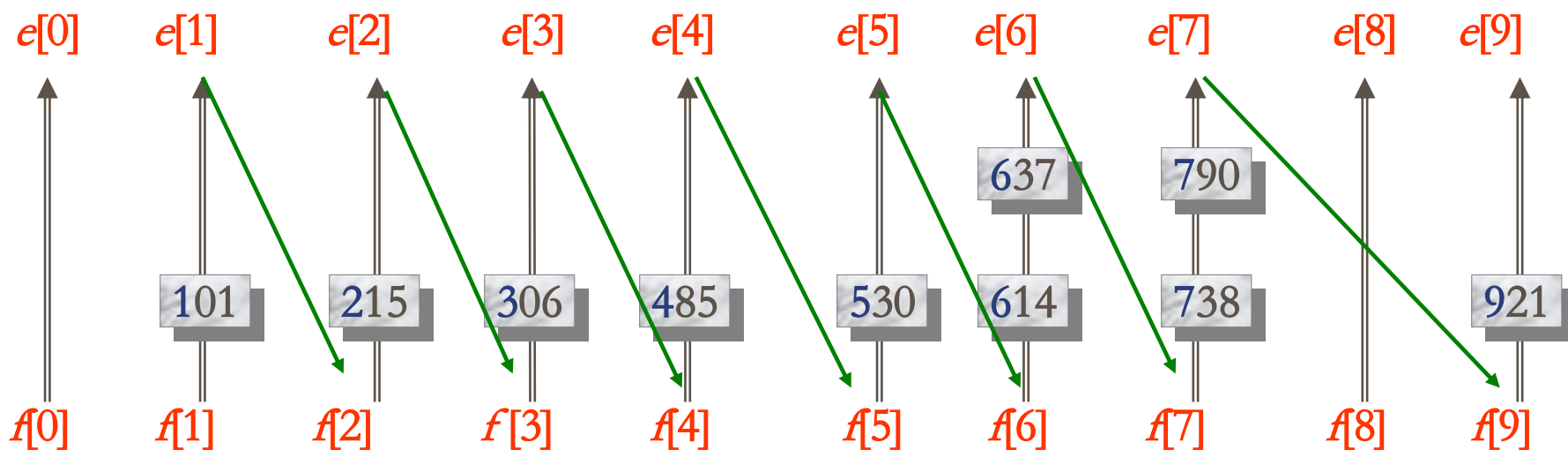


10.6 基数排序

第二趟收集的结果:



第三趟分配 (按最高位 $i=1$)



第三趟收集 (让队尾指针 $e[i]$ 链接到下一非空队首指针 $f[i+1]$)

排序结束!



10.6 基数排序

链式基数排序

如何编程实现？

1. 用静态链表形式存储排序前后的 n 个记录（在 $r[]$ 中增开 n 个指针分量）；这样在记录重排时不必移动数据，只需修改各记录的链接指针即可。

10.6 基数排序

链式基数排序

如何编程实现？

2. 在 radix 个静态队列中，每个队列都要设置两个指针：

$\text{int } f[\text{radix}]$ 指示队头 ($f[j]$ 初始为空) ；

$\text{int } e[\text{radix}]$ 指向队尾 ($e[j]$ 可省略初始化, 队尾元素随时赋值)；

分配到同一队列的关键码要用链接指针链接起来。

(注：以上一共增开了 $n + 2 \times \text{radix}$ 个附加指针分量)

10.6 基数排序

链式基数排序

如何编程实现？

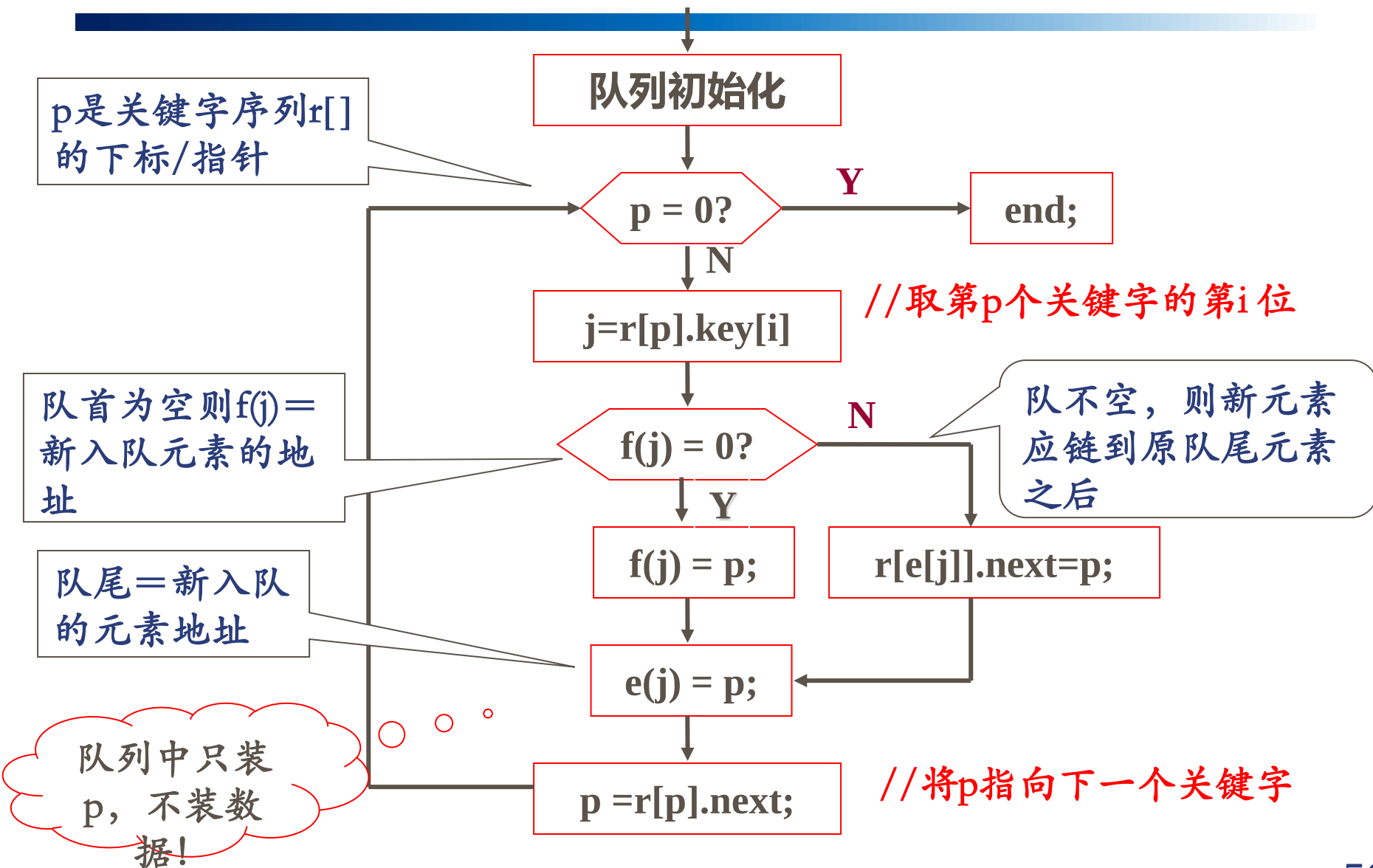
3. 存入 $r[]$ 中的待排记录可分为3部分：



第 p 个记录的关键码的第 i 位
可表示为： $r[p].key[i]$

第 p 个记录的指针分量
可表示为： $r[p].next$

一趟“分配”过程的算法流程



10.6 基数排序

```
void RadixSort(SLList &L){ // L是采用静态链表表示的顺序表
// 基数排序,使得L成为按关键字自小到大有序,L.r[0]为头结点
    for (i = 0; i < L.recnum; ++i)
        L.r[i].next = i + 1;
    L.r[recnum].next = 0; // 将L改造为静态链表
    for (i = 0; i < L.keynum; ++i)
    { // 按最低位优先依次对各关键字进行分配和收集
        Distribute(L.r, i, f, e); // 第i趟分配
        Collect(L.r, i, f, e);    // 第 i 趟收集
    }
} // RadixSort
```

10.6 基数排序

```
void Distribute (SLCell &r,int i, ArrType &f,ArrType &e){  
    //静态链表L的r域中记录已按(keys[0], ..keys[i- 1])有序。  
    //本算法按第i个关键字keys[i]建立 RADIX个子表使同一子表中记录的 keys[i]相同  
    // f[0...RADIX-1]和e[0...RADIX-1]分别指向各子表中第一个和最后一个记录  
    for(j=0;j<Radix; ++j) f[j] = 0; //各子表初始化为空表  
    for (p=r[0].next; p; p=r[p].next) {  
        j = ord(r[p].keys[i]); // ord将记录中第i个关键字映射到[0...RADIX-1]  
        if(!f[j]) f[j]=p;  
        else r[e[j]].next = p;  
        e[j]=p; // 将p所指的结点插入第 j个子表中  
    }  
}
```



10.6 基数排序

```
void Collect (SLCell &.r, int i, ArrType f, ArrType e) {  
    //本算法按keys[i]自小至大地将f[0...RADIX-1]所指各子表依次链接成一个链表  
    //e[0...RADIX- 1]为各子表的尾指针  
    for(j=0; !f[j]; j=succ(j)); //找第一个非空子表, succ 为求后继函数  
    r[0].next = f[j]; // r[0].next 指向第一个非空子表中第一个结点  
    t=e[j];  
    while(j<RADIX){  
        for (j=succ(j); j<RADIX-1 && !f[j]; j= succ(j)); // 找下一个非空子表  
        if(f[j]) {r[t].next = f[j]; t=e[j];} //链接两个非空子表  
    }  
    r[t].next = 0; // t 指向最后一个非空子表中的最后一个结点  
}
```

10.6 基数排序

链式基数排序

基数排序算法分析

- 假设有 n 个记录, 每个记录的关键字有 d 位, 每个关键字的取值有 $radix$ 个, 则每趟分配需要的时间为 $O(n)$, 每趟收集需要的时间为 $O(radix)$, 合计每趟总时间为 $O(n+radix)$

全部排序需要重复进行 d 趟“分配”与“收集”。因此时间复杂度为: $O(d(n+radix))$ 。

10.6 基数排序

链式基数排序

基数排序算法分析

- 基数排序需要增加 $n+2radix$ 个附加链接指针，空间效率低，
空间复杂度： $O(n+radix)$.
- 稳定性： 稳定。(一直前后有序)。

小结

排序

一、时间性能

1. 按平均的时间性能来分，有三类排序方法

- 时间复杂度为 $O(n\log n)$ 的方法有
 - 快速排序、堆排序和归并排序，其中以快速排序为最好
- 时间复杂度为 $O(n^2)$ 的有
 - 直接插入排序、冒泡排序和简单选择排序，其中以直接插入为最好，特别是对那些对关键字近似有序的记录尤为如此
- 时间复杂度为 $O(n)$ 的排序方法只有：
 - 基数排序

排序

一、时间性能

2. 当待排序记录序列按关键字顺序有序时，直接插入排序和冒泡排序能达到 $O(n)$ 的时间复杂度。而对于快速排序而言，这是最不好的情况，此时的时间性能退化为 $O(n^2)$ ，因此应该避免这种情况
3. 简单选择排序、堆排序和归并排序的时间性能不随记录序列中关键字的分布而该改变

排序

二、空间性能（指的是排序过程中所需的辅助空间大小）

1. 所有的简单排序方法(包括: 直接插入, 冒泡和简单选择)和堆排序的空间复杂度为 $O(1)$
2. 快速排序为 $O(\log n)$, 为栈所需的辅助空间
3. 归并排序所需辅助空间最多, 其空间复杂度为 $O(n)$
4. 链式基数排序需附设队列首尾指针, 则空间复杂度为 $O(n + \text{radix})$

排序

三、排序算法稳定性

- 稳定的排序方法是指，对于两个关键字相等的记录，它们在序列中的相对位置，在排序之前和经过排序之后，没有改变。
- 当对多关键字的记录序列进行LSD方法排序时，必须采用稳定的排序方法
- 对于不稳定的排序方法，只要能举出一个实例说明即可
- 快速排序和堆排序是不稳定的排序方法

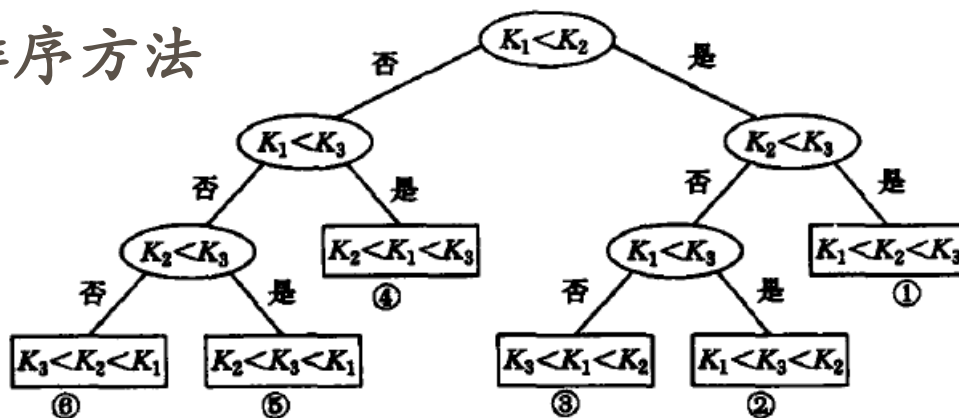
排序

四、关于“排序方法的时间复杂度的下限”

- 本章各种排序方法，除基数排序外，其他方法都是基于“比较关键字”进行排序的排序方法。可以证明，这类排序法可能达到的最快的时间复杂度为 $O(n\log n)$

(基数排序不是基于“比较关键字”的排序方法，所以它不受这个限制)

- 可以用一棵判定树来描述这类基于“比较关键字”进行排序的排序方法



排序

排序方法	时间复杂度	时间复杂度		稳定性	附加存储	
		最好	最差		最好	最差
直接插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	√	$O(1)$	
折半插入排序	$O(n\log_2 n)$	$n\log_2 n$		√	$O(1)$	
2-路插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	√	$O(1)$	
表插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	√	$O(n)$	
希尔排序	$O(n^{1.25})$	$O(n)$	$O(1.6n^{1.25})$	X	$O(1)$	
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	√	$O(1)$	
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	X	$O(\log_2 n)$	$O(n^2)$
简单选择排序	$O(n^2)$	n^2		X	$O(1)$	
树形选择排序	$O(n\log_2 n)$	$O(n\log_2 n)$		√	$O(n)$	
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$		X	$O(1)$	
归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$		√	$O(n)$	
基数排序	$O(d(n+\text{radix}))$	$O(d(n+\text{radix}))$		√	$O(n+\text{radix})$	